

Actividad 12

Laboratorio: File Path Traversal, Validation of File Extension with Null Byte Bypass

PortSwigger Web Security Academy

Elaborado por: Juan Pablo Casanova De La Madrid

Herramienta utilizada: Burp Suite Community Edition

1. Nombre del laboratorio

File path traversal, validation of file extension with null byte bypass

Laboratorio perteneciente a PortSwigger Web Security Academy, dentro de la categoría File path traversal.

2. Nivel

Practitioner.

En este laboratorio, la aplicación valida que el nombre de archivo recibido termine con la extensión esperada (por ejemplo, .png o .jpg), como control adicional para restringir el tipo de recurso que puede solicitarse. Evadir esta validación conservando al mismo tiempo el acceso a un archivo sin esa extensión (como /etc/passwd) requiere una técnica distinta a las empleadas en los laboratorios anteriores, basada en la inyección de un byte nulo.

3. Tipo de explotación

Path Traversal / Directory Traversal mediante bypass de validación de extensión con byte nulo (Null Byte Injection) (CWE-22: Improper Limitation of a Pathname to a Restricted Directory, relacionado con CWE-158: Improper Neutralization of Null Byte or NUL Character).

Es una vulnerabilidad del lado del servidor que permite a un atacante manipular la ruta de un archivo solicitado para acceder a archivos fuera del directorio raíz previsto por la aplicación. En este laboratorio, la explotación aprovecha que la aplicación valida la extensión del nombre de archivo utilizando funciones de comparación de cadenas de alto nivel, mientras que el sistema operativo, al abrir el archivo, interpreta un byte nulo (%00) como el final efectivo de la cadena, ignorando todo lo que le sigue.

4. Objetivo del laboratorio

El laboratorio contiene una vulnerabilidad de path traversal en la funcionalidad que muestra las imágenes de los productos de la tienda. La aplicación valida que el nombre de archivo proporcionado termine con la extensión de archivo esperada.

Objetivo: explotar dicha vulnerabilidad añadiendo un byte nulo seguido de una extensión válida al final de una ruta de traversal hacia /etc/passwd, de forma que la validación de extensión se supere en la capa de la aplicación, mientras que el sistema operativo acceda realmente al archivo /etc/passwd, y así dar por resuelto el laboratorio.

Lab: File path traversal, validation of file extension with null byte bypass

PRACTITIONER



LAB

Not solved



This lab contains a path traversal vulnerability in the display of product images.

The application validates that the supplied filename ends with the expected file extension.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

Figura 1. Descripción y objetivo del laboratorio en PortSwigger Web Security Academy (nivel Practitioner, estado Not solved).

5. Explicación técnica de la vulnerabilidad

La aplicación web muestra el catálogo de productos ("We Like To Shop") y cada imagen se sirve dinámicamente mediante un endpoint del tipo:

```
GET /image?filename=<nombre_de_archivo>
```

Como mecanismo de protección, la aplicación valida que el valor recibido en filename termine con una extensión de archivo permitida (por ejemplo, .png), normalmente comprobando si la cadena finaliza con ese sufijo. Si la condición no se cumple, la petición es rechazada.

El problema es que esta validación se realiza a nivel de aplicación, sobre la cadena de texto completa, mientras que la operación real de apertura del archivo se ejecuta en una capa inferior (llamadas de sistema del sistema operativo o de una biblioteca en C) que interpreta el carácter nulo (byte 0x00, representado en la URL como %00) como el terminador de la cadena. Esto significa que, para la aplicación, la cadena ".././etc/passwd%00.png" termina en ".png" y por lo tanto es válida, pero para el sistema operativo, la cadena efectiva a abrir termina en el byte nulo, es decir, ".././etc/passwd", ignorando por completo el sufijo ".png".

Como resultado, es posible construir una ruta de traversal que retrocede desde el directorio de imágenes hasta la raíz del sistema de archivos y referencia /etc/passwd, añadiendo al final un byte nulo seguido de la extensión esperada únicamente para satisfacer la validación de la aplicación. Esta vulnerabilidad se clasifica dentro de CWE-22 y CWE-158, y es un ejemplo clásico de discrepancia (mismatch) entre cómo distintas capas de un sistema interpretan una misma cadena de texto.

6. Procedimiento paso a paso

1. Se inicia Burp Suite Community Edition, seleccionando "Temporary project in memory" y la configuración por defecto ("Use Burp defaults").

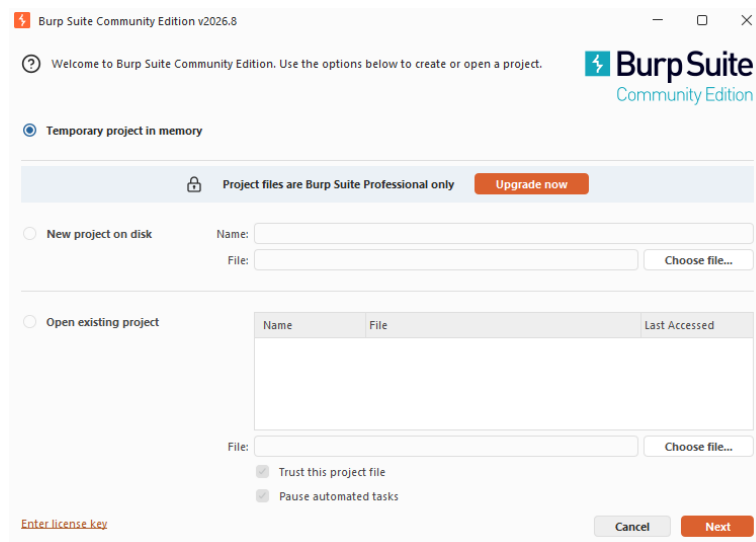


Figura 2. Creación de un proyecto temporal en memoria.

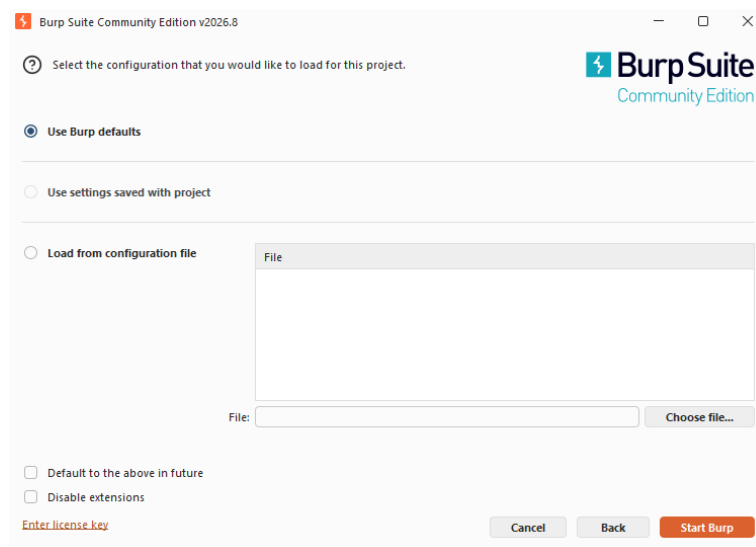


Figura 3. Selección de la configuración por defecto de Burp.

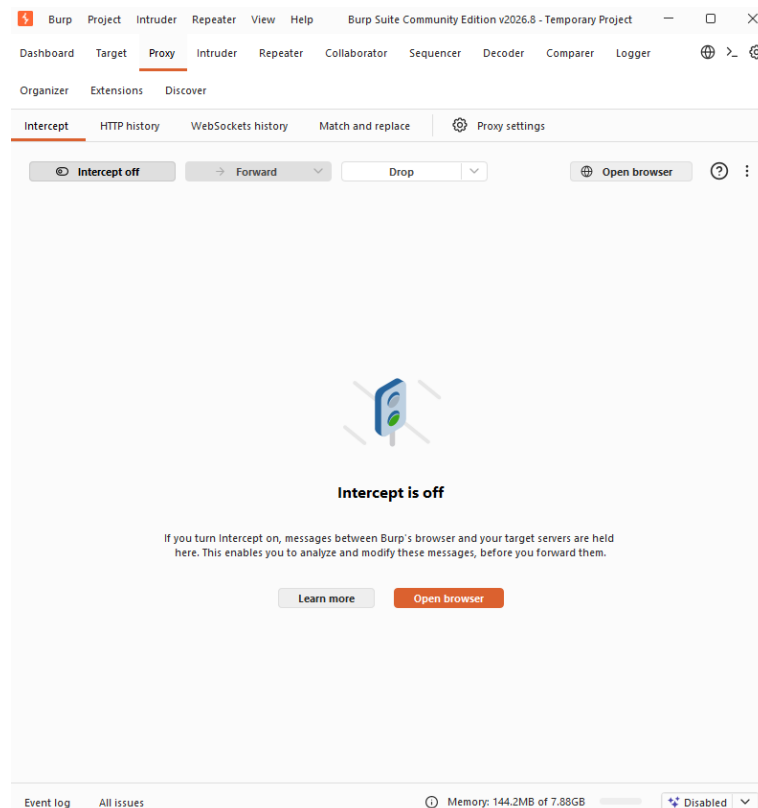


Figura 4. Burp Suite iniciado, pestaña Proxy con Intercept inicialmente apagado.

1. Se accede al laboratorio desde la Web Security Academy de PortSwigger, con el tráfico pasando a través del navegador integrado de Burp.

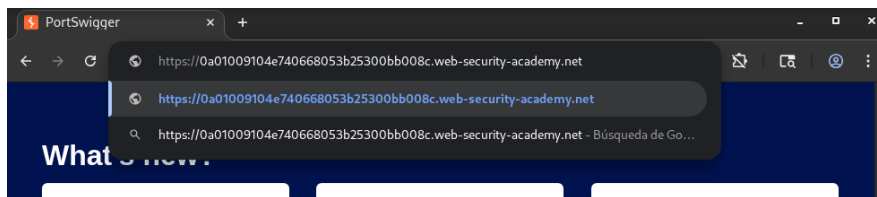


Figura 5. Navegación hacia la URL del laboratorio a través del navegador de Burp.

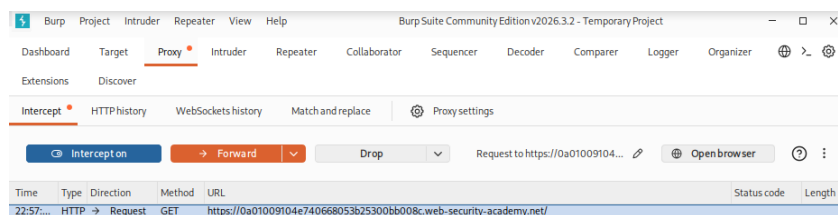


Figura 6. Petición GET inicial capturada en el historial del Proxy al abrir el laboratorio.

- Con el Intercept activo, se navega por la tienda; Burp captura las peticiones GET que la página realiza para cargar cada imagen de producto, del tipo /image?filename=NN.jpg, confirmando que el nombre de archivo se maneja mediante un parámetro controlable por el cliente.

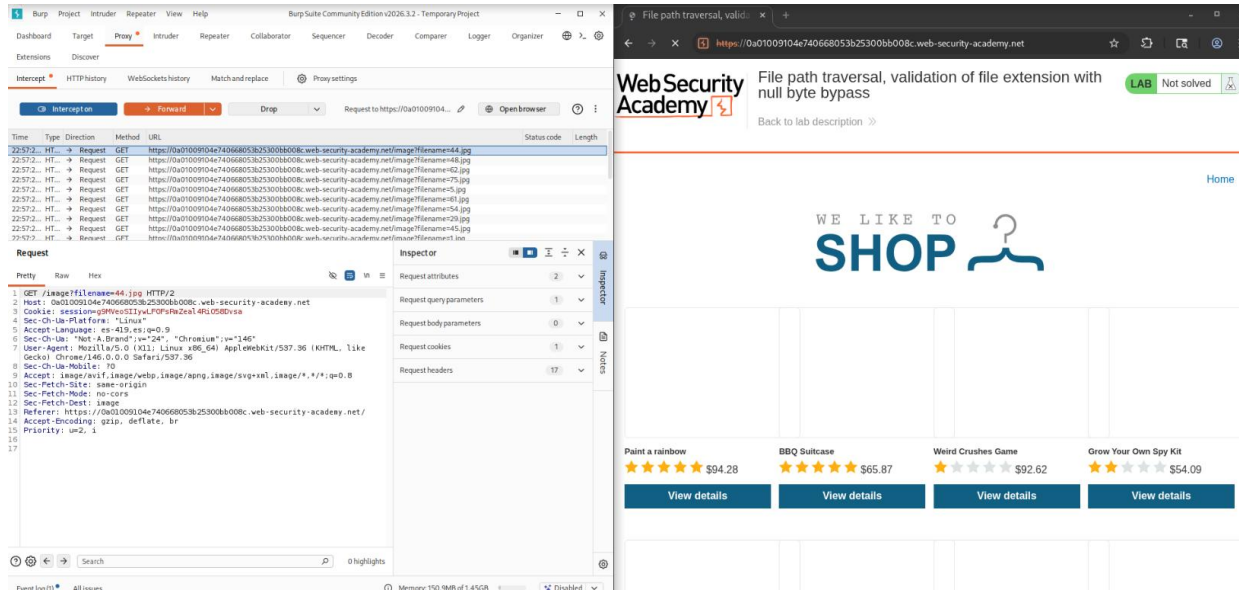


Figura 7. Petición interceptada GET /image?filename=44.jpg junto con el resto de las peticiones de imágenes en el historial del Proxy.

- Se da clic en "View details" de uno de los productos; Burp intercepta la petición GET /product?productId=1 correspondiente.

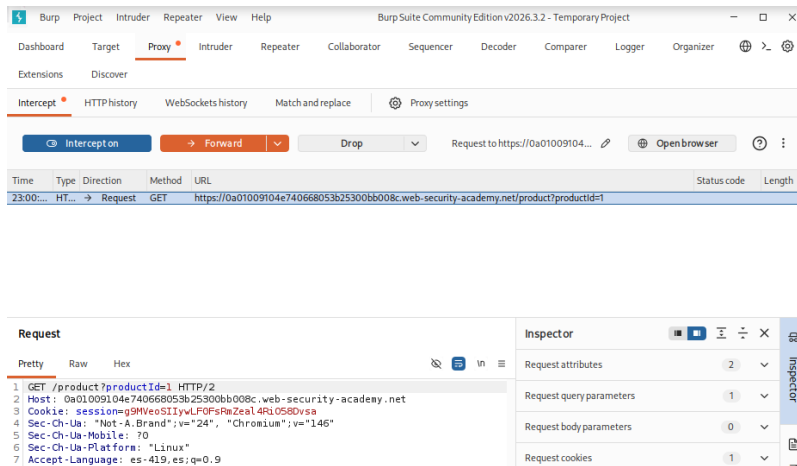


Figura 8. Petición GET /product?productId=1 interceptada tras dar clic en "View details".

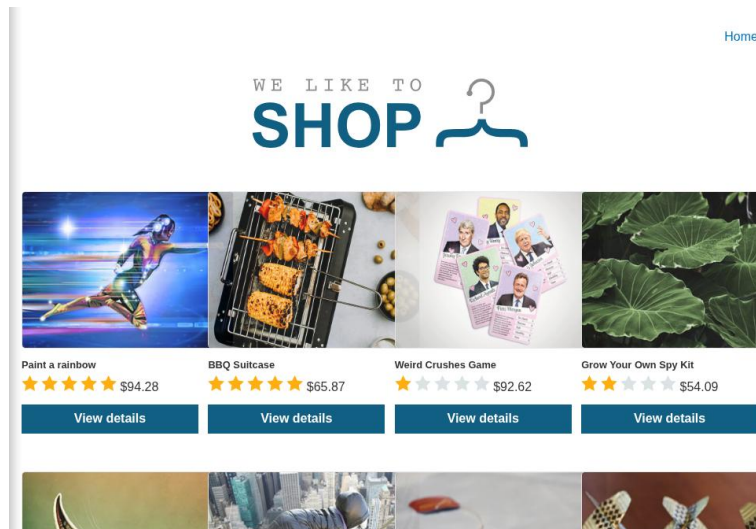


Figura 9. Vista de la tienda mostrando los productos cuyas imágenes se cargan mediante el parámetro filename.

- Una vez capturada la petición GET /image?filename=44.jpg correspondiente a la imagen del producto, se envía al módulo Repeater ("Send to Repeater") para poder modificarla y reenviarla libremente.

Request

1 GET /image?filename=44.jpg HTTP/2

2 Host: 0a01009104e740668053b25300bb008c.web-security-academy.net

3 Cookie: session=g9MVe0SIIYwLF0FsRmZeal4Ri058Dvsa

4 Sec-Ch-Ua-Platform: "Linux"

5 Accept-Language: es-419,es;q=0.9

6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"

7 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

8 Sec-Ch-Ua-Mobile: ?0

9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8

10 Sec-Fetch-Site: same-origin

11 Sec-Fetch-Mode: no-cors

12 Sec-Fetch-Dest: image

13 Referer: https://0a01009104e740668053b25300bb008c.web-security-academy.net/product?productId=1

14 Accept-Encoding: gzip, deflate, br

15 Priority: u=2, i

16

17

Inspector

Request attributes 2

Request query parameters 1

Request body parameters 0

Request cookies 1

Request headers 17

Figura 10. Petición GET /image?filename=44.jpg interceptada y enviada a Repeater.

5. En Repeater, se sustituye 44.jpg por la secuencia de traversal ../../etc/passwd, agregando al final un byte nulo codificado (%00) seguido de la extensión .png, con el fin de superar la validación de extensión de la aplicación.
GET /image?filename=../../etc/passwd%00.png HTTP/2

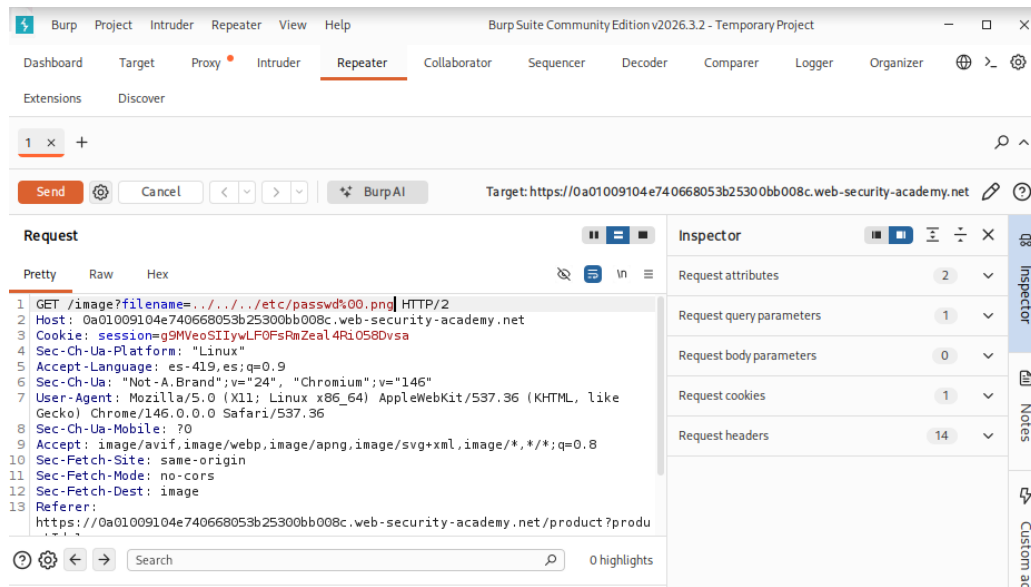


Figura 11. Payload con inyección de byte nulo inyectado en el parámetro filename.

6. Se hace clic en Send y se analiza la respuesta del servidor.
7. El servidor responde con código HTTP/2 200 OK y devuelve en el cuerpo de la respuesta el contenido íntegro del archivo /etc/passwd, confirmando la explotación exitosa de la vulnerabilidad.

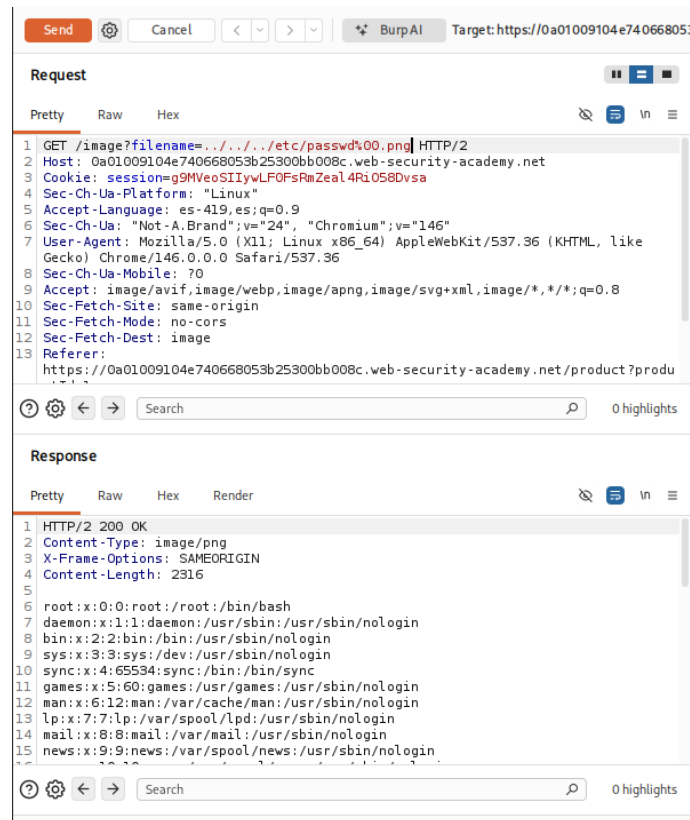


Figura 12. Petición con el payload y respuesta vulnerable (200 OK) mostrando el contenido de `/etc/passwd`.

- Finalmente, se regresa a la página del laboratorio en PortSwigger, la cual actualiza automáticamente su estado a "Solved", confirmando que el objetivo de la actividad fue completado.

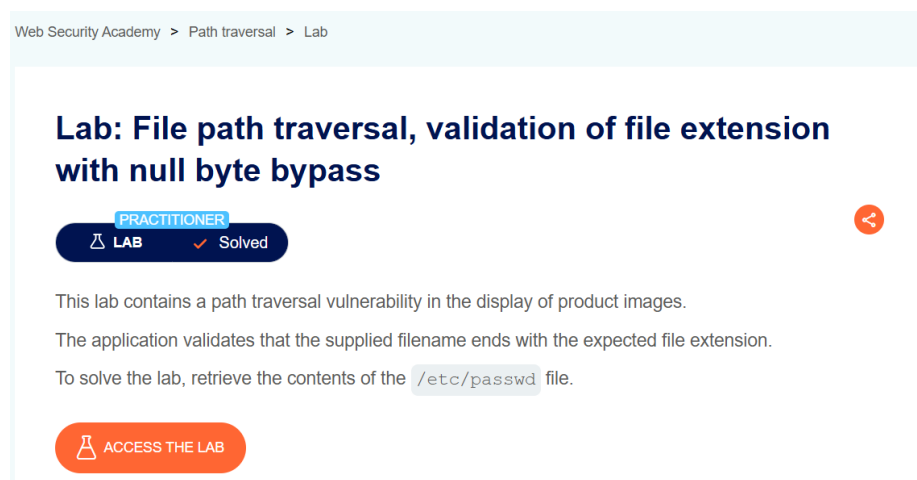


Figura 13. Confirmación de laboratorio resuelto ("Solved").

7. Explicación del payload

GET `/image?filename=../../../../etc/passwd%00.png` HTTP/2

El payload se construye a partir de tres componentes que trabajan en conjunto:

- **../../../../**: secuencia de retroceso de directorios que permite salir del directorio de imágenes y llegar a la raíz del sistema de archivos, de la misma forma que en un ataque de path traversal simple.
- **etc/passwd**: ruta del archivo objetivo que se desea leer, ya ubicado desde la raíz del sistema de archivos.
- **%00.png**: %00 es la representación codificada en URL del byte nulo (carácter con valor 0x00). Al añadirlo justo después de "passwd" y antes de la extensión ".png", se logra que la cadena completa, evaluada por la lógica de validación de la aplicación (normalmente una comprobación de tipo "¿termina la cadena en .png?"), se considere válida, ya que en apariencia el nombre de archivo finaliza con la extensión permitida.

Sin embargo, cuando esta cadena llega a las funciones del sistema operativo o de bibliotecas de bajo nivel encargadas de abrir el archivo, estas interpretan el byte nulo como el carácter de terminación de la cadena (tal como ocurre en el lenguaje C), por lo que todo lo que aparece después de %00 (en este caso, ".png") es ignorado. El archivo que efectivamente se abre es "../../../../etc/passwd", sin ninguna extensión adicional. En síntesis, el payload no evita la detección del traversal, sino que engaña a la validación de extensión aprovechando que dos capas distintas del sistema interpretan el final de la misma cadena de forma diferente.

8. Mitigación recomendada

Para prevenir vulnerabilidades de path traversal como la explotada en este laboratorio, se recomienda aplicar las siguientes medidas, de forma combinada:

- Actualizar y mantener parcheado el entorno de ejecución (lenguaje, runtime y bibliotecas del sistema), ya que la mayoría de los frameworks y lenguajes modernos ya rechazan explícitamente los bytes nulos dentro de nombres de archivo antes de realizar operaciones de sistema de archivos.
- Rechazar de forma explícita cualquier valor de entrada que contenga el carácter nulo (%00 o su representación decodificada) en el parámetro filename, antes de realizar cualquier otra validación.
- Validar el parámetro filename contra una lista blanca (whitelist) de nombres de archivo o identificadores permitidos, en lugar de depender únicamente de una comprobación de sufijo/extensión sobre la cadena recibida.
- Resolver la ruta final de forma canónica (canonicalización) y verificar que el resultado permanezca dentro del directorio base autorizado, como comprobación adicional e independiente de la validación de extensión.
- Evitar exponer directamente el sistema de archivos del servidor a partir de datos proporcionados por el usuario; en su lugar, mapear identificadores de producto/imagen a rutas físicas mediante una tabla o índice controlado por el backend.
- Ejecutar el proceso de la aplicación con el mínimo privilegio necesario, restringiendo los permisos de lectura del usuario del servicio web únicamente a los directorios estrictamente necesarios.